

complexity. Hence, the overall complexity is $O(N)$, the time complexity of the first step which dominates the overall algorithm's time taken. Note that as K becomes larger and close to N , retrieval of top K elements can end up being as costly as $O(N \log N)$. What would be some of the test cases you need to think about while you consider different approaches? Some obvious test cases include empty array of integers, and boundary values of K such as 1 and N . You may also want to look at a test case where all numbers are identical in the input array.

Let us briefly consider one more problem.

- (2) *Given an array of N binary data elements of 1s and 0s, what is the minimum number of swaps needed to group all the 1s together, with the cluster of 1s starting at any place in the array?*

Again, let us follow an iterative approach similar to what we discussed earlier. What is the simple and naïve solution we can come up with for this problem? Since there are N possible positions in the array, we can consider clustering the 1s at each of these locations; then computing the number of swaps for each of them and finding the minimum among them. This can be done naively with an $O(N^2)$ time complexity. Now, how can we improve this solution? What are the ways in which you can break down this problem?

- (a) We need to identify the index locations in the array where the cluster of 1s can be placed. Is this N as we considered earlier, or can we reduce it to a smaller number?
- (b) How can we efficiently compute the number of 1s in each of the positions identified in the first step?

If you have a single '1' only in the array, there is no swap needed. If we have K 1s in the array, what are all the possible positions for the start of this cluster? It can be anywhere from 1 to $N-K$ (since you need space to hold the K 1s in the array). Now, at each of these $N-K$ positions, there is a window of elements currently present in the array. Instead of the current set of elements in each of these windows, we want to put the cluster of 1s in that window. How many swaps are needed to achieve this?

A window of ' K ' elements contains a mix of 1s and 0s. You don't need to do anything to an element if it has a value of 1. If it has a value of 0, you need to swap out that element and bring a 1 in its location. Once you finish doing this operation for all the K elements in the window, the window will contain a cluster of K 1s. Hence you can now rephrase this as follows.

Instead of counting the number of swaps needed to cluster the 1s in each window, this problem can be reduced to counting the number of 0s in each window. You may be asking yourself by now, how does this reduce the complexity from $O(N^2)$ to $O(N)$?

Well, any time you want to look for means to reducing the time complexity, think about how you can reuse previously computed results. If you have computed the number of 0s in a K -length window starting at ' I ' already and want to compute the same for the next window starting at $K+1$, how can you reuse the results from the last calculation? The current window will include all the elements from the previous window except the $A[I]$ element, and it will include a new element at $A[I+1+K]$. If the element $A[I]$ was a 0, then you decrement the previous result by 1. If the new element $A[I+1+K]$ is a 0, then increment the previous result by 1. Hence, by reusing the previously computed results, you can reduce the time complexity from $O(N^2)$ to $O(N)$.

What are the various test cases you should consider for this problem? Consider an array of all 1s, 0 1s, 1s and 0s alternating, a single '1' present in the array and also a case where all 1s are already clustered in the array. It is also important to think about the variations of a given problem. That will allow you to quickly modify solutions of known problems to unseen problems. What are the possible variations to this problem?

- (1) Instead of binary data in the array, how will you solve this problem if you have ternary data? Essentially, you are given an N element integer array consisting of either 0, 1 or 2 as the value of elements in the array. What is the minimum number of swaps needed to cluster all 1s in the array at any place in the array?
- (2) A variation of the above is to restrict the starting point of the cluster. How would your solution change if you want to cluster all the 1s at the beginning of the array (index 0)?
- (3) Instead of allowing swaps to any location, we are now considering the case where you can swap only adjacent elements. Given an array of N binary data elements of 1s and 0s, what is the minimum number of swaps needed to group all the 1s together, with the cluster of 1s starting at any place in the array? You can only swap adjacent elements in the array.

Feel free to reach out to me over LinkedIn/email if you need any help in your new study journey. If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Wishing all our readers happy coding until next month! Take care, and stay healthy and safe. 

 By: Sandya Mannarswamy

The author is an expert in natural language processing and is currently working as an independent researcher. Her interests include natural language processing, machine learning and AI.